

Names: David Fatheree, Preston May

MAIN/TOP SECTION OF VERILOG CODE

```
`timescale 1ns / 1ps

module top(
    input    i_rx,
    input    clk,
    output    o_tx,
    input    rst
);

    // UART signals
    reg    tx_start;
    wire    rx_valid, tx_done, tx_busy;
    reg [127:0] tx_byte;
    wire [255:0] rx_byte;

    // UART module
    uart_sm #(.BAUD(115200), .CLK_SPEED(100_000_000)) UART_LINK (
        .clk(clk),
        .reset(rst),
        .i_rx(i_rx),
        .o_tx(o_tx),
        .tx_start(tx_start),
        .tx_done(tx_done),
        .tx_busy(tx_busy),
        .rx_done_really(rx_valid),
        .rx_words(rx_byte),
        .tx_words(tx_byte)
    );

    // FSM states
    localparam RST    = 2'b00,
        WAIT    = 2'b01,
        COMPUTE = 2'b10,
        START_TX = 2'b11;

    reg [1:0] state;
```

```

reg [127:0] plaintext_reg;
reg [127:0] key_in;
wire [127:0] ciphertext;

wire [127:0] k0, k1, k2, k3, k4, k5, k6, k7, k8, k9, k10;

key_generator u_keygen (
    .key_in(key_in),
    .k0(k0), .k1(k1), .k2(k2), .k3(k3), .k4(k4),
    .k5(k5), .k6(k6), .k7(k7), .k8(k8), .k9(k9), .k10(k10)
);

wire [127:0] r0_out, r1_out, r2_out, r3_out, r4_out, r5_out, r6_out, r7_out, r8_out, r9_out,
r10_out;

assign r0_out = plaintext_reg ^ k0;

aes_round r1 (.clk(clk), .in(r0_out), .key(k1), .out(r1_out));
aes_round r2 (.clk(clk), .in(r1_out), .key(k2), .out(r2_out));
aes_round r3 (.clk(clk), .in(r2_out), .key(k3), .out(r3_out));
aes_round r4 (.clk(clk), .in(r3_out), .key(k4), .out(r4_out));
aes_round r5 (.clk(clk), .in(r4_out), .key(k5), .out(r5_out));
aes_round r6 (.clk(clk), .in(r5_out), .key(k6), .out(r6_out));
aes_round r7 (.clk(clk), .in(r6_out), .key(k7), .out(r7_out));
aes_round r8 (.clk(clk), .in(r7_out), .key(k8), .out(r8_out));
aes_round r9 (.clk(clk), .in(r8_out), .key(k9), .out(r9_out));

aes_final_round r10 (.clk(clk), .in(r9_out), .key(k10), .out(r10_out));

assign ciphertext = r10_out;

// State-Machine logic
always @(posedge clk or posedge rst) begin
    if (rst) begin
        state <= RST;
        tx_start <= 0;
        tx_byte <= 0;
        plaintext_reg <= 0;
        key_in <= 0;
    end else begin
        tx_start <= 0;
        case (state)
            RST: begin

```

```

        if (rx_valid) begin
            plaintext_reg <= rx_byte[127:0];
            key_in    <= rx_byte[255:128];
            state <= COMPUTE;
        end
    end

    COMPUTE: begin
        tx_byte <= ciphertext;
        state <= START_TX;
    end

    START_TX: begin
        if (!tx_busy) begin
            tx_start <= 1'b1;
            state <= WAIT;
        end
    end

    WAIT: begin
        if (tx_done)
            state <= RST;
        end
    endcase
end
end
endmodule

```

PYTHON TEST CODE

```
import sys, time, serial, random
from AES_Python import AES
```

```
SERIAL_PORT = "COM3"
BAUD_RATE = 115200
NUM_TESTS = 20
```

```
SBOX = [
0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfe,0xd7,0xab,0x76,
0xca,0x82,0xc9,0x7d,0xfa,0x59,0x47,0xf0,0xad,0xd4,0xa2,0xaf,0x9c,0xa4,0x72,0xc0,
0xb7,0xfd,0x93,0x26,0x36,0x3f,0xf7,0xcc,0x34,0xa5,0xe5,0xf1,0x71,0xd8,0x31,0x15,
0x04,0xc7,0x23,0xc3,0x18,0x96,0x05,0x9a,0x07,0x12,0x80,0xe2,0xeb,0x27,0xb2,0x75,
0x09,0x83,0x2c,0x1a,0x1b,0x6e,0x5a,0xa0,0x52,0x3b,0xd6,0xb3,0x29,0xe3,0x2f,0x84,
0x53,0xd1,0x00,0xed,0x20,0xfc,0xb1,0x5b,0x6a,0xcb,0xbe,0x39,0x4a,0x4c,0x58,0xcf,
0xd0,0xef,0xaa,0xfb,0x43,0x4d,0x33,0x85,0x45,0xf9,0x02,0x7f,0x50,0x3c,0x9f,0xa8,
0x51,0xa3,0x40,0x8f,0x92,0x9d,0x38,0xf5,0xbc,0xb6,0xda,0x21,0x10,0xff,0xf3,0xd2,
0xcd,0x0c,0x13,0xec,0x5f,0x97,0x44,0x17,0xc4,0xa7,0x7e,0x3d,0x64,0x5d,0x19,0x73,
0x60,0x81,0x4f,0xdc,0x22,0x2a,0x90,0x88,0x46,0xee,0xb8,0x14,0xde,0x5e,0x0b,0xdb,
0xe0,0x32,0x3a,0x0a,0x49,0x06,0x24,0x5c,0xc2,0xd3,0xac,0x62,0x91,0x95,0xe4,0x79,
0xe7,0xc8,0x37,0x6d,0x8d,0xd5,0x4e,0xa9,0x6c,0x56,0xf4,0xea,0x65,0x7a,0xae,0x08,
0xba,0x78,0x25,0x2e,0x1c,0xa6,0xb4,0xc6,0xe8,0xdd,0x74,0x1f,0x4b,0xbd,0x8b,0x8a,
0x70,0x3e,0xb5,0x66,0x48,0x03,0xf6,0x0e,0x61,0x35,0x57,0xb9,0x86,0xc1,0x1d,0x9e,
0xe1,0xf8,0x98,0x11,0x69,0xd9,0x8e,0x94,0x9b,0x1e,0x87,0xe9,0xce,0x55,0x28,0xdf,
0x8c,0xa1,0x89,0x0d,0xbf,0xe6,0x42,0x68,0x41,0x99,0x2d,0x0f,0xb0,0x54,0xbb,0x16
]
```

```
RCON = [0x01,0x02,0x04,0x08,0x10,0x20,0x40,0x80,0x1B,0x36]
```

```
def sub_bytes(state):
    return bytes(SBOX[b] for b in state)
```

```
def shift_rows(state_bytes):
    s = list(state_bytes)
    m = [
        [s[0], s[4], s[8], s[12]], # row 0
        [s[1], s[5], s[9], s[13]], # row 1
        [s[2], s[6], s[10], s[14]], # row 2
```

```

    [s[3], s[7], s[11], s[15]], # row 3
]
out = []
for i, row in enumerate(m):
    out.extend(row[i:] + row[:i])
return bytes(out)

def transpose_state(state):
    out = bytearray(16)
    for r in range(4):
        for c in range(4):
            out[4 * r + c] = state[4 * c + r]
    return bytes(out)

def mul(a, b):
    res = 0
    for _ in range(8):
        if b & 1:
            res ^= a
        high = a & 0x80
        a = (a << 1) & 0xFF
        if high:
            a ^= 0x1B
        b >>= 1
    return res

def mix_single_column(col):
    a = list(col)
    return bytes([
        mul(2,a[0]) ^ mul(3,a[1]) ^ a[2] ^ a[3],
        a[0] ^ mul(2,a[1]) ^ mul(3,a[2]) ^ a[3],
        a[0] ^ a[1] ^ mul(2,a[2]) ^ mul(3,a[3]),
        mul(3,a[0]) ^ a[1] ^ a[2] ^ mul(2,a[3])
    ])

def mix_columns(state):
    s = [list(state[i::4]) for i in range(4)]
    mixed = [mix_single_column(col) for col in s]
    out = bytearray(16)
    for c in range(4):
        for r in range(4):
            out[r + 4*c] = mixed[c][r]
    return bytes(out)

```

```

def sub_word(w):
    return ((SBOX[(w>>24)&0xFF]<<24)|
            (SBOX[(w>>16)&0xFF]<<16)|
            (SBOX[(w>>8)&0xFF]<<8)|
            SBOX[w&0xFF])

def rot_word(w):
    return ((w<<8)&0xFFFFFFFF)|(w>>24)

def expand_key(key):
    w = [int.from_bytes(key[i:i+4], 'big') for i in range(0, 16, 4)]
    for i in range(4, 44):
        temp = w[i - 1]
        if i % 4 == 0:
            temp = sub_word(rot_word(temp)) ^ (RCON[(i // 4) - 1] << 24)
        w.append(w[i - 4] ^ temp)
    return [b''.join(x.to_bytes(4, 'big') for x in w[i:i+4]) for i in range(0, 44, 4)]

def aes_encrypt(plaintext, key):
    round_keys = expand_key(key)
    state = bytes(a ^ b for a, b in zip(plaintext, round_keys[0]))
    for r in range(1, 10):
        state = sub_bytes(state)
        state = shift_rows(state)
        state = mix_columns(state)
        state = bytes(a ^ b for a, b in zip(state, round_keys[r]))

    # Final round w/ internal transpose
    state = sub_bytes(state)
    state = shift_rows(state)
    state = transpose_state(state)
    state = bytes(a ^ b for a, b in zip(state, round_keys[10]))
    return state

def send_and_receive(port, baud, key, plaintext):
    data = key + plaintext
    with serial.Serial(port, baud, timeout=1.0) as ser:
        ser.reset_input_buffer()
        ser.reset_output_buffer()
        time.sleep(0.02)
        ser.write(data)
        ser.flush()
        return ser.read(16)

```

```

def main():
    print(f"Testing AES-128 via UART on {SERIAL_PORT} @ {BAUD_RATE}")
    total = 0
    passed = 0
    for i in range(NUM_TESTS):
        key=bytes(random.getrandbits(8) for _ in range(16))
        plaintext=bytes(random.getrandbits(8) for _ in range(16))

        expected = aes_encrypt(plaintext, key)

        rx = send_and_receive(SERIAL_PORT, BAUD_RATE, key, plaintext)

        print(f"\nTest {i+1:02d}")
        print(f" Key      : {key.hex()}")
        print(f" Plaintext  : {plaintext.hex()}")
        print(f" Expected   : {expected.hex()}")
        ok = (len(rx) == 16 and rx == expected)
        if len(rx) != 16:
            print(f" Received   : (incomplete, {len(rx)} bytes) {rx.hex()}")
        else:
            print(f" Received : {rx.hex()}")
        print(" → PASS" if ok else " → FAIL")
        total += 1
        passed += ok
    print(f"\n{passed}/{NUM_TESTS} tests passed")

if __name__ == "__main__":
    main()

```

RESULTS SNAPSHOT

```
134
135 rx = send_and_receive(SERIAL_PORT, BAU
136
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\defdo\Documents\part1> python part5b.py

→ PASS

Test 17

Key : aaef8241ff3d4badbbf876aa7de1f686
Plaintext : d69f378c3a96904bc0899ec981077369
Expected : 52c9fad576a34a0be13f5ddfc6b69894
Received : 52c9fad576a34a0be13f5ddfc6b69894
→ PASS

Test 18

Key : e195bbce0c5594957a3f57d77c07495e
Plaintext : 559d9ddb085e9b6f13125cdcd1158c34
Expected : d84d1d6669100f970b33aa6566420e6c
Received : d84d1d6669100f970b33aa6566420e6c
→ PASS

Test 19

Key : fb6d8acc78bedd293020e91c93ea25d0
Plaintext : d7b3d56d77a820a69266df7ee70e5518
Expected : 888d3da9da2a53fdc5e918090fb06ef0
Received : 888d3da9da2a53fdc5e918090fb06ef0
→ PASS

Test 20

Key : c93cdc38f50e224b6db8f6dc964e1345
Plaintext : e6b3a26bbaa0ddf097a5b9934cb2321c
Expected : 248939f856dab864a43753213f827200
Received : 248939f856dab864a43753213f827200
→ PASS